

Part 2: TPUs

Q1) 200B parameters bfloat split over 32 TPU v4p

$$400B \text{ bytes} = 4.00e11 \text{ bytes} = (4.00e11 / 32) \text{ bytes per chip}$$

$$\text{TPU v4p HBM BW} = 1.2e12 \text{ bytes/second}$$

~~transmission~~

$$t_{\text{trans per chip}} = \frac{4.00e11 \text{ bytes}}{32 \times 1.2e12 \text{ bytes/second}} = 1.04e-2 \text{ seconds}$$

$$t_{\text{trans}} = 1e3 \text{ ns}$$

$$1e3 \text{ ns} = 1 \mu\text{s}$$

10 ms

Q2) Full TPU v5e pod

$$\text{Total CPU hosts} = \text{num chips} / \text{chips per host} = (16 \times 16) / (4 \times 2)$$

$$\text{Num TPU cores} = \text{chips} \times 1 \text{ for v5e} = 256$$

$$= 256 / 8 = 32$$

$$\text{Total FLOPs/s} = \text{chips} \times \text{FLOPs/s per chip} = 256 \times 1.97e14$$

$$\text{Total HBM} = 256 \times 8.2e11 = 2.10e14$$

$$= 5.04e16 \text{ FLOPs/s}$$

(6 float 16)

$$\text{HBM} = 16 \times 256 = 4TB$$

Full TPU v5p pod

$$\text{Total Hosts} = \frac{16 \times 20 \times 28}{2 \times 2 \times 1} = 2240 \text{ hosts}$$

$$\text{Num Cores} = 16 \times 20 \times 28 \times 2 = 17920 \text{ cores}$$

$$\text{Total FLOPs/s} = (16 \times 20 \times 28) \times 4.59e14 = 4.11e18$$

$$\text{Total HBM} = 8960 \times 2.8e12 = 2.51e16$$

per chip

(Q3) PCIe BW = 1.6×10^4 bytes/second, A is $\text{bfib}[D, F]$
 X is $\text{bfib}[B, D]$

let $B < D$ and $F = 4D$

Optimize on TPU vbe, so Intensity (accelerator) = $\frac{9.20 \times 10^4 \text{ FLOPs/s}}{1.6 \times 10^4 \text{ bytes/s}} = 57500$

(since

t_{comm} will be limited by PCIe)

$$\text{Intensity (memory)} = \frac{BF(2D-1)}{2BD + DDF + BF} = \frac{B(4D)(2D-1)}{2BD + DDF + BF}$$

$$= \frac{B(4D)(2D-1)}{2D(1+4D)} = \frac{2B}{2} = B$$

for $B \geq 57,500$, memory becomes compute bound

(Q4) (TPU v2e)

Int 8 [8, 4096] • Int 8 [4096, 16384], where B is unknown

1. How long will this take?

Int 8s needed = $B(2(4096) - 1)(16384) = 1.34 \times 10^8 B$

bytes to load = $B(8 \times 4096) + (16384 \times 4096) + 16384 B$
 $= 1.34 \times 10^8 + 2.04 \times 10^8 B$ bytes

~~so $t_{\text{comm}} = \frac{1.34 \times 10^8 B}{1.34 \times 10^8 + 2.04 \times 10^8 B}$~~

~~$t_{\text{comm}} = \frac{1.34 \times 10^8 + 2.04 \times 10^8 B}{8.2 \times 10^4 \text{ bytes/second}}$~~

~~$t_{\text{comm}} = 1.6 \times 10^{-4} \text{ seconds} + 2.49 \times 10^{-7} B$~~

~~$t_{\text{memory}} = \max(t_{\text{comm}}, t_{\text{math}})$~~

$t_{\text{math}} = \frac{1.34 \times 10^8 \text{ ops}}{3.94 \times 10^4 \text{ FLOPs/s}}$

$t_{\text{math}} = 3.40 \times 10^{-7} B$

VMEM takes about 22×10^3 time than HBM. So

$$t_{comm,vmem} = \frac{t_{comm,hbm}}{22} = \frac{1.4e-4 + 2.49e-7B}{22}$$

then take $\max(t_{comm,vmem}, t_{math})$

Q4) TRVise:

Int 8 [B, 4096]. Int 8 [4096, 16384]

$$t_{math} = \frac{\text{total int ops}}{3.94e14 \text{ ops/s}} = \frac{2(B)(4096)(16384)}{3.94e14} = \frac{1.34e8B}{3.94e14}$$

$$t_{comm} = \frac{\text{total bytes/writes}}{8.2e11 \text{ bytes/second}} = \frac{(4096)(16384) + B(4096) + B(16384)}{8.2e11}$$

$$t_{min} = \max(t_{math}, t_{comm})$$

$$\text{Intensity (accelerator)} = \frac{3.94e14}{8.2e11} = 480 \text{ so we want}$$

$$\text{Min } 1.34e8B$$

$$\frac{(4096)(16384) + 4096B + 16384B}{8.2e11} \geq 480$$

$$\begin{aligned} 1.34e8B - (480)4096B - (480)(16384B) &\geq (480)(16384)(4096) \\ &= B(1.34e8 - 2e6 - 8e6) \geq 32e9 \\ &= 1.24e8B \geq 32e9, \text{ so } B \geq \frac{32e9}{1.24e8} \approx 258 \checkmark \end{aligned}$$

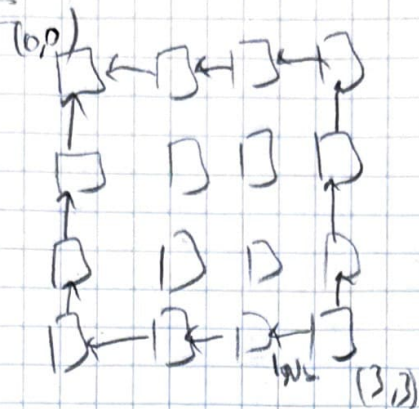
If we look from VMEM, take $\frac{258}{22} \approx 11.7$ so $B \geq 11$

Q5) TPO v5e 4x4 size.

→ 1 core, 1 app. ~~4 chips~~ 2 times of 4 chips/host

We want to send from TPO {0,0} to TPO {3,3}. Per hop latency = 10s

No wrap around



$$\text{BW/Link LCI} = 4.5 \text{ Gbps}$$

$$\text{Total bytes} = 2 \times 128 \times 8 \times 8192$$

$$= 16,777,216 \text{ bytes}$$

$$t_{\text{stream}} = 3.72 \times 10^{-4} \text{ seconds}$$

$$t_{2\text{stream}} = \frac{3.72 \times 10^{-4}}{2} = 1.86 \times 10^{-4} \text{ seconds}$$

$$t_{\text{first byte}} = 6 \mu\text{s} \quad \checkmark$$

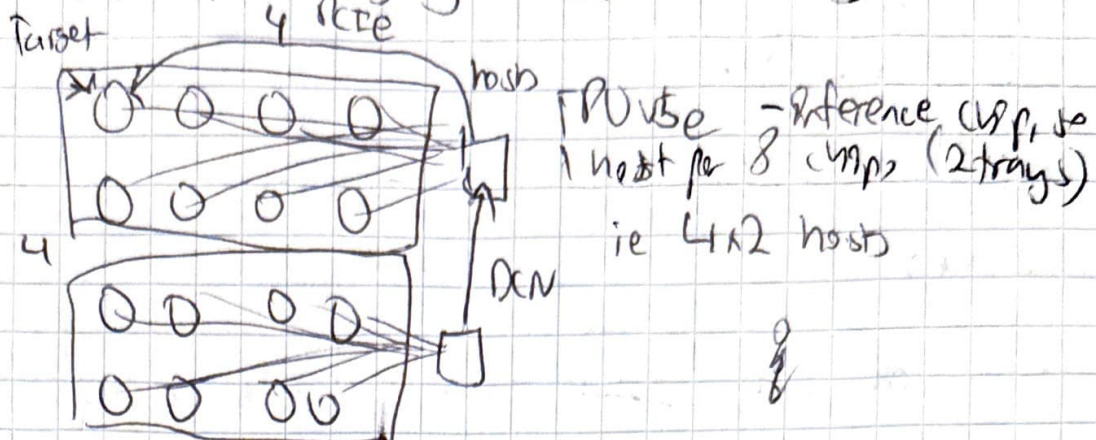
$$t_{\text{total byte}} = 6 \mu\text{s} + 166 \mu\text{s} = 172 \mu\text{s} \quad \checkmark$$

bit 2-

Question 6

Matrix A: 128×124 , $128, 124$, sharded evenly over 16 GPUs
 4×4 slice, but on host RAM. We want to copy to ~~GPU~~
 $\{0, 2\}$, multiply by $16 \times 128 \times 124$.

A).



How to transfer data: HBM, ICI, PCIe, PCN.

HBM: TensorCore to HBM

ICI: Between GPU chips and neighbours

PCIe: Between GPU host and its tray (16 chips)

PCN: Between multiple GPU hosts, typically not connected by ICI.

so have to hop to go travel long distances.

Strategy II

We have PCN as $3.125 \times 10^9 \times 8 = 2.5 \times 10^{10}$ bytes/second,

so we have to transfer ~~128~~ ~~128~~ ~~128~~ ~~128~~ bytes

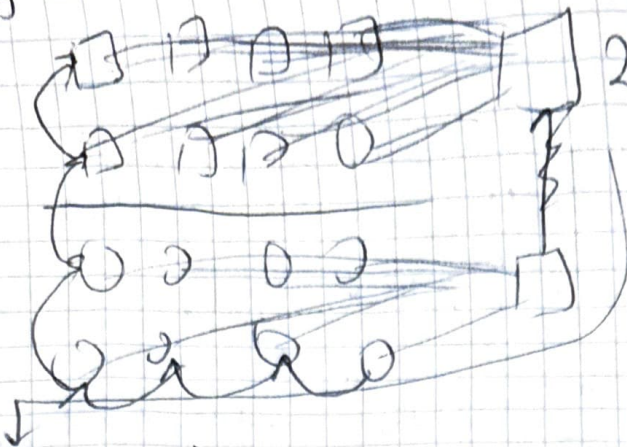
$$t_{comm, PCN} = \frac{128 \times 124 \times 128 \times 16 \text{ GiB}}{2.5 \times 10^{10} \text{ bytes/second}} = 5.5 \times 10^{-5} \text{ s}$$

$$t_{comm, PCIe} = \frac{128 \times 124 \times 128 \times 16 \text{ GiB}}{1.6 \times 10^{10} \text{ bytes/second}} \approx 1.07 \text{ seconds!!}$$

$$t_{comm, HBM} \text{ will be the fastest, so } t_{comm} \text{ for HBM to VMEM is } \frac{16 \text{ GiB}}{3.94 \times 10^4} \approx 0.34 \text{ ms}$$

* remember to check dtype per matrix!

Strategy 2:



1. Load evenly across all P2P
2. Hop to $\{0,0\}$ over ECI.

$$= \frac{16 \text{ GiB (header)}}{2 \times 10^6 \text{ b/s}} = 0.000008 \text{ sec} = 8 \text{ ns}$$

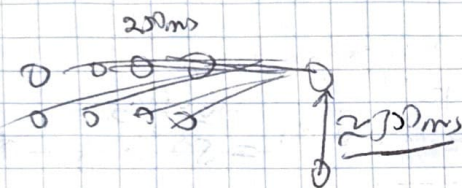
to Hop, we need 6 hops ^{at max}, ~~so that is~~, each transferring $\frac{16 \text{ GiB}}{16}$ bytes,

$$\text{so } t_{\text{comm}} = \frac{16 \text{ GiB} \cdot 6}{16 \times 4 \cdot 5 \cdot 10^6} \text{ bc we do not have a full } 16 \text{ ring.}$$

$$= 0.143 \text{ sec} = 143 \text{ ms}$$

$143 \text{ ms} > 67 \text{ ms}$, so we are dominated by $t_{\text{comm}} \text{ ECI}$.

So, the operation should take on the order of 143 ms



$$t_{\text{all}} \geq \max(t_{\text{comm}}, t_{\text{mem}})$$

$$t_{\text{all}} \leq \text{sum}(t_{\text{comm}} + t_{\text{mem}})$$

so assuming good overlapping,

143 ms is not a close!

Optimiz.

$$\text{we have } 8.128 \cdot 4.175 \text{ sec} = 70 \text{ PLOPs/s}$$